# CC2: Argumentation Theory and Applications

Aleyna Yagbasan, Kenza Abou El Kasem, Mariam El Ouarrad

October 18, 2026

## Introduction

Argumentation theory studies how conclusions can be drawn from arguments that conflict with each other. In this course, we saw several ways to do this: structured argumentation, where arguments are built from rules and assumptions; argument mining, where arguments and their relations are extracted from natural language; and gradual semantics, where each argument receives a degree of acceptability instead of being simply accepted or rejected. This report covers these three aspects, one in each part.

In Part 1, we implement a generator for Assumption-Based Argumentation (ABA). It reads a framework, tests whether it is circular and atomic, converts it into a non-circular and atomic framework, builds all the arguments and attacks, and takes preferences into account with the normal and reverse attacks of ABA+. We check our results against the correction of TD4, and we deploy the generator as an online application.

In Part 2, we move from formal frameworks to real debates. We scrape 250 debates from the Kialo platform, explore the data, and fine-tune a RoBERTa model to decide whether two arguments are related and, if so, whether one supports or attacks the other. We evaluate the model on debates it has never seen and compare it with results from the literature.

In Part 3, we implement the weighted h-categorizer and use it to visualise acceptability degree spaces: for a given graph, we draw many random weight vectors and look at all the degrees the arguments can reach. With convex hulls, heat-maps and a few metrics, we study how the structure of the graph limits these degrees, and what we can still say about the space when there are too many arguments to draw it.

**Code and application.** All the code is in the GitHub repository `https://github.com/aleynaygb/dyco` and in the ZIP file submitted with this report, with one folder per part.

## 1 Part 1: Reasoning with ABA

**Files.** The generator is `src/aba.py` (classes `Rule` and `ABAFramework`); running `python3 src/aba.py` reproduces all the results below. The application uses three more files: `app.py`, `input_parser.py` and `requirements.txt`. `aba.py` only needs Python 3 and its standard library; the application also needs Streamlit (see `requirements.txt`). The notebook `src/aba_generator.ipynb` shows the results.

### 1.1 Question 1: the ABA generator

Everything is in one class, `ABAFramework`, plus a small class `Rule`. We test it on the framework of TD4 (circular and not atomic) and on a small umbrella example.

#### 1.1.1 Literals, rules, assumptions and contraries

An ABA framework is $(\mathcal{L}, \mathcal{R}, \mathcal{A}, {}^-)$. In the class:

- `literals` (set).
- `rules` (list of `Rule(head, body)`; the body is a tuple and can be empty, which is a fact).
- `assumptions` (set).
- `contraries` (dict assumption $\to$ contrary).
- `preferences` (set of pairs (weaker, stronger)).

`add_assumption(a, c)` puts `a` and its contrary `c` in $\mathcal{L}$, so an assumption always has a contrary.

**Example, the framework of TD4:** $\mathcal{L} = \{a, b, x, y, z\}$, $\mathcal{A} = \{a, b\}$, $\overline{a} = y$, $\overline{b} = x$, $\mathcal{R} = \{y \leftarrow b;\ y \leftarrow y;\ x \leftarrow x;\ x \leftarrow a;\ z \leftarrow x, y\}$. Figure 1 shows how we define it with the class and what the code prints.

```python
f = ABAFramework()

# L
f.add_literal("x")
f.add_literal("y")
f.add_literal("z")

# A + contraries
f.add_assumption("a", "y")
f.add_assumption("b", "x")

# R
f.add_rule("y", ["b"])
f.add_rule("y", ["y"]) #cycle
f.add_rule("x", ["x"]) #cycle
f.add_rule("x", ["a"])
f.add_rule("z", ["x","y"])
```

```
=== original framework ===
literals    : ['a', 'b', 'x', 'y', 'z']
assumptions : ['a', 'b']
rules       :
    y <- b
    y <- y
    x <- x
    x <- a
    z <- x, y
contraries  :
    contrary(a) = y
    contrary(b) = x
```

Figure 1: The TD4 framework: definition with `ABAFramework` (left) and printed framework (right).

### 1.1.2 Atomic and circular tests

A rule is atomic if its body contains only assumptions, and a framework is atomic if all its rules are.

- `is_atomic()` checks body $\subseteq \mathcal{A}$ for each rule.
- `is_circular()` builds a dictionary head $\rightarrow$ literals of the bodies and looks for a cycle with a depth-first search that remembers the current path.

On TD4 we get circular = `True` and atomic = `False`.

### 1.1.3 Conversion to non-circular and atomic frameworks

Two things slow down the construction of the arguments: loops ($y \leftarrow y$) and chains ($z \leftarrow x, y$). We remove them in two steps, always in this order, because step 2 is only valid on a non-circular framework: `convert()` calls `to_non_circular()` and then `to_atomic()`.

**Step 1, non-circular.** We make $k = |\mathcal{L} \setminus \mathcal{A}|$ levels of each non-assumption ($x_1, x_2, x$; the last level keeps the original name). A rule of level $i$ uses only literals of level $i-1$, so we can only go up and never loop. Atomic rules are copied at levels $1..k$ and the other rules at levels $2..k$. We apply it to any framework, because the theorem of the course says the extensions are preserved.

```python
def to_non_circular(self):
    non_assump = self.literals - self.assumptions
    k = len(non_assump) # number of levels

    def name(s, i):
        # last level keeps the original name
        return s if i == k else f"{s}_{i}"

    new = self.copy_without_rules()
    # add the copies of each non assumption (one per level)
    for s in non_assump:
        for i in range(1, k + 1):
            new.literals.add(name(s, i))

    for rule in self.rules:
        atomic = set(rule.body) <= self.assumptions
        start = 1 if atomic else 2 # atomic rules: levels 1..k, others: 2..k
        for i in range(start, k + 1):
            if atomic:
                new_body = rule.body
            else:
                # assumptions stay, other literals come from the level below
                new_body = tuple(b if b in self.assumptions else name(b, i - 1)
                                 for b in rule.body)
            new.rules.append(Rule(name(rule.head, i), new_body))
    return new
```

Figure 2: `to_non_circular()`: one copy of each non-assumption per level.

**Step 2, atomic.** For each non-assumption $s$ we create two new assumptions: $s_d$ ("$s$ is derived") and $s_{nd}$ ("$s$ is not derived"), with $\overline{s_d} = s_{nd}$ and $\overline{s_{nd}} = s$. In every rule body, $s$ is replaced by the bet $s_d$. The bet is attacked by $s_{nd}$, but if $s$ can really be derived, then $s$ attacks $s_{nd}$, so $s_d$ is defended. If $s$ cannot be derived, nothing defends $s_d$.

```python
def to_atomic(self):
    non_assump = self.literals - self.assumptions
    new = self.copy_without_rules()
    # two new assumptions for each non assumption
    for s in non_assump:
        new.literals |= {s + "_d", s + "_nd"}
        new.assumptions |= {s + "_d", s + "_nd"}
        new.contraries[s + "_d"] = s + "_nd"
        new.contraries[s + "_nd"] = s
    # in each body, a non assumption s becomes s_d
    for rule in self.rules:
        new_body = tuple(b if b in self.assumptions else b + "_d" for b in rule.body)
        new.rules.append(Rule(rule.head, new_body))
    return new
```

Figure 3: `to_atomic()`: each non-assumption $s$ of a body is replaced by the new assumption $s_d$.

The new assumptions $s_d$ and $s_{nd}$ get no preference: the preferences stay those given by the user.
**Result on TD4** ($k = 3$: number of literals that are not assumptions $(x, y, z)$), in Table 1:

| Framework | circular | atomic | rules | assumptions |
|---|---|---|---|---|
| original | True | False | 5 | 2 |
| `to_non_circular()` | False | False | 12 | 2 |
| `convert()` | False | True | 12 | 20 |

Table 1: TD4 framework before and after each conversion.

The loops became staircases: $y \leftarrow y$ gives $y_2 \leftarrow y_1$ and $y \leftarrow y_2$, and in the atomic framework the first one becomes $y_2 \leftarrow y_{1_d}$. The 12 rules, the 20 assumptions and the contraries are identical to the correction of TD4 (checked by a program, after removing the "_" of our names, for example `x_1_d` becomes `x1d`).

### 1.1.4 Arguments and attacks

An argument is a pair $(X, s)$: $X$ is the set of assumptions it uses and $s$ its conclusion, written $\{X\} \vdash s$. $(X, s)$ attacks $(Y, t)$ if $s$ is the contrary of an assumption of $Y$. `build_arguments` is a fixpoint: every assumption $a$ gives $(\{a\}, a)$; for each rule we take one support for each literal of the body, merge them and get a new argument for the head; we repeat until nothing new appears. We store pairs (support, conclusion) and not trees, so the loop stops even on a circular framework. `build_attacks` compares every pair of arguments. Table 2 gives the results on TD4.

```python
def build_arguments(self):
    # an argument is (support, conclusion)
    # every assumption is an argument for itself
    args = {(frozenset([a]), a) for a in self.assumptions}
    changed = True
    while changed: # repeat until nothing new
        changed = False
        for rule in self.rules:
            # for each literal of the body, all the supports that prove it
            options = [[X for (X, s) in args if s == b] for b in rule.body]
            # pick one support per body literal and merge them
            combos = [frozenset()]
            for opt in options:
                combos = [c | X for c in combos for X in opt]
            for X in combos:
                if (X, rule.head) not in args:
                    args.add((X, rule.head))
                    changed = True
    return args
```

Figure 4: `build_arguments()`: a fixpoint on pairs (support, conclusion).

| TD4 framework | Arguments | Attacks |
|---|---|---|
| original (circular) | 5 | 6 |
| non-circular | 10 | 12 |
| atomic (`convert()`) | 32 | 45 |

Table 2: Arguments and attacks of the TD4 framework.

The 5 arguments of the original framework are $\{a, b\} \vdash z$, $\{a\} \vdash a$, $\{a\} \vdash x$, $\{b\} \vdash b$, $\{b\} \vdash y$. The non-circular framework has 5 more, with the conclusions $x_1, x_2, y_1, y_2, z_2$. Its 10 arguments and 12 attacks are those of the correction of TD4.

### 1.1.5 Preferences: normal and reverse attacks

We follow the ABA+ framework of the course: an ABA framework with a transitive preference relation $\leq$ on the assumptions ($x < y$ means that $x$ is less preferred than $y$). An argument with support $X'$ and conclusion the contrary of an assumption $y$ of $Y$ gives a **normal attack** from $X'$ to $Y$ if no $x'$ in $X'$ satisfies $x' < y$. If there is such an $x'$, the attack is **reversed**: $Y$ attacks $X'$.

```python
def build_attacks_plus(self, args):
    normal, reverse = set(), set()
    for (X, s) in args:
        for (Y, t) in args:
            for y in Y:
                if self.contraries[y] == s: # (X, s) attacks y
                    # is one assumption of x weaker than the attacked y ?
                    if any((x, y) in self.preferences for x in X):
                        reverse.add(((Y, t), (X, s))) # attack is reversed
                    else:
                        normal.add(((X, s), (Y, t))) # normal attack
    return normal, reverse
```

Figure 5: `build_attacks_plus()`: normal and reverse attacks.

In the code, we work on arguments instead of sets of assumptions (their supports are the sets of the definition). The argument $(X, s)$ attacks $(Y, t)$ through $y$ in $Y$ when $s$ is the contrary of $y$. The attack is normal if no assumption of $X$ is less preferred than $y$, and reversed otherwise, so $(Y, t)$ attacks $(X, s)$.

**Example, the umbrella.** The forecast says rain and the sky looks sunny. `forecast_ok` and `sky_ok` are assumptions with $\overline{\texttt{forecast_ok}} = \texttt{sun}$ and $\overline{\texttt{sky_ok}} = \texttt{rain}$. Rules: `rain` $\leftarrow$ `forecast_ok`, `sun` $\leftarrow$ `sky_ok`, `umbrella` $\leftarrow$ `rain`. We trust the forecast more than our eyes: `sky_ok` $<$ `forecast_ok`. The framework and the results printed by the code are in Figure 6.

```
literals    : ['forecast_ok', 'rain', 'sky_ok', 'sun', 'umbrella']
assumptions : ['forecast_ok', 'sky_ok']
rules       :
    rain <- forecast_ok
    sun <- sky_ok
    umbrella <- rain
contraries  :
    contrary(forecast_ok) = sun
    contrary(sky_ok) = rain
preferences : [('sky_ok', 'forecast_ok')]
atomic      : False
circular    : False
```

```
attacks without preferences : 5
normal attacks : 2
    {forecast_ok} |- rain --> {sky_ok} |- sky_ok
    {forecast_ok} |- rain --> {sky_ok} |- sun
reverse attacks : 3
    {forecast_ok} |- forecast_ok --> {sky_ok} |- sun
    {forecast_ok} |- rain --> {sky_ok} |- sun
    {forecast_ok} |- umbrella --> {sky_ok} |- sun
```

Figure 6: The umbrella example: framework (left) and attacks (right).

Without preferences the two camps attack each other (5 attacks). With the preference only the forecast camp attacks: its 2 attacks stay normal, and the 3 attacks of $\{\texttt{sky_ok}\} \vdash \texttt{sun}$ are reversed, because its support contains the weaker `sky_ok`. So we take the umbrella. The rule `umbrella` $\leftarrow$ `rain` is not atomic; `to_atomic()` turns it into `umbrella` $\leftarrow$ `rain_d`.

### 1.1.6 Critical evaluation

**What works.** The generator does everything asked on the TD4 example, and its results are the same as the correction of the TD (12 rules, 20 assumptions, 10 arguments and 12 attacks after the non-circular conversion). The preferences also behave as expected: in the umbrella example, only the forecast side attacks once we trust it more than the sky.

**What it costs.** The conversion makes the framework much bigger: $|\mathcal{A}'| = |\mathcal{A}| + 2 \cdot k \cdot |\mathcal{L} \setminus \mathcal{A}|$. For TD4, 2 assumptions become 20, and for the example of Question 2 ($k = 5$), 3 assumptions become 53 and 5 rules become 22. The number of arguments grows too, because `build_arguments` combines one support for each literal of a body: the same example goes from 8 to 75 arguments after conversion.

**What it does not do.** The generator builds the arguments and attacks, but it does not compute extensions (admissible, preferred, ...), which would be the next step to decide which assumptions to accept.

## 1.2 Question 2: the online application

**Link of the application:** `https://dyco.onrender.com/`

The application is written with Streamlit and deployed on Render (free web service). The free instance goes to sleep after 15 minutes without visit, so the first loading of the page can take about one minute: please wait.

**How to use it.**

1. Open the link.
2. Write the framework in the text box (or keep the example already written), in the format below.
3. Click on **Generate**. The page shows: (1) the framework with its properties, (2) the non-circular framework, (3) the non-circular and atomic framework, (4) all the arguments, (5) all the attacks, (6) the normal and reverse attacks. Arguments and attacks are computed on the framework that was typed.
4. If the input is wrong, a red message explains the error (for example "no contrary given for c" or "a is an assumption, it cannot be the head of a rule").

**Input format.** (An example is already written in the box):

- `L: [a,b,...]` all the literals, in square brackets as in the statement;
- `A: [a,...]` the assumptions (they must be in L);
- `C(a): r` the contrary of `a` (one line per assumption);
- `[r1]: p <- q,a` is a rule head `<-` body (body separated by commas). An empty body is a fact (`[r2]: q <-`). The name `r1` is not used.
- `PREF: a > b`: `a` is preferred to `b`. `a,b > c`: `a` and `b` are equally preferred and both preferred to `c`. The line is optional. Lines starting with `##` and empty lines are ignored.

**Result.** For the input of the statement, the application gives 8 arguments and 10 attacks, 6 normal and 4 reverse (Figure 7). The 4 reverse attacks come from $\{b, c\} \vdash r$: its support contains $b$ and $b < a$, so its attacks on the arguments that use $a$ are turned around and now point to $\{b, c\} \vdash r$. The attacks of $\{a, c\} \vdash s$ and $\{a, c\} \vdash t$ on $\{b, c\} \vdash r$ appear in both lists: once as normal attacks (on $b$ and on $c$), once as reversed ones (from the attacks of $r$ on $a$). The framework is not circular and not atomic; with

$k = 5$ the conversion gives 53 assumptions, 22 rules and 75 arguments (computed with `convert()` and `build_arguments()` on this input). The converted frameworks are too long to show here; they can be viewed in the application (sections 2 and 3).

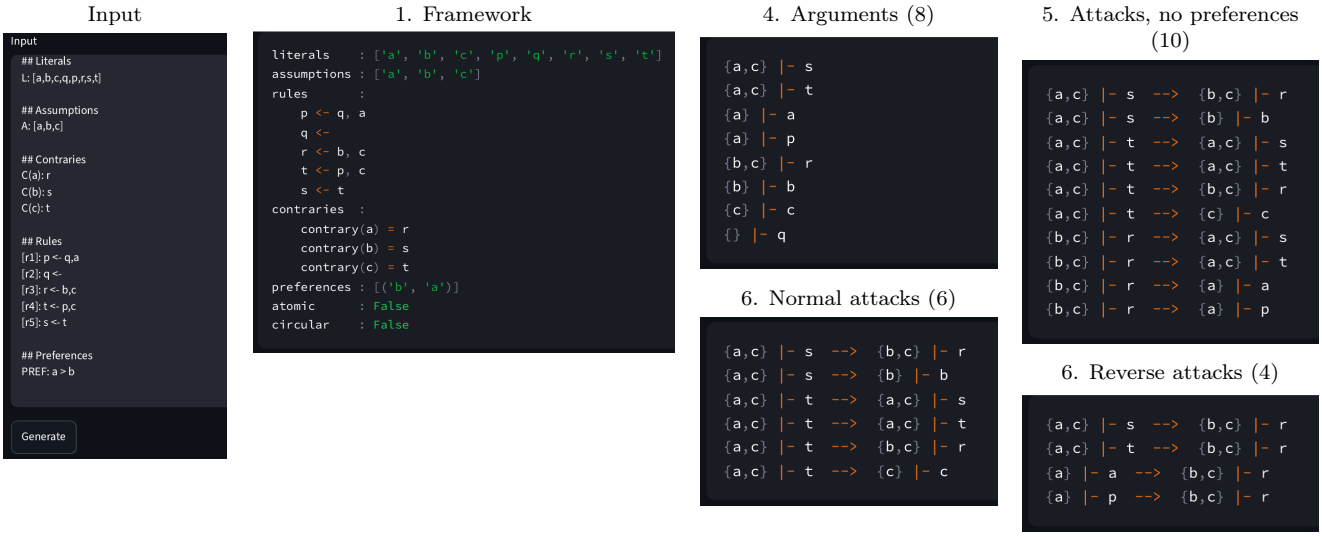



Figure 7: The application on the input of the statement: input, framework, arguments, attacks, and normal/reverse attacks.

**Deployment.** `app.py` is the page, `input_parser.py` reads and checks the text and builds an `ABAFramework`, and `aba.py` is the generator. The files are in a GitHub repository (folder `src`). Render settings: Root Directory `src`; Build Command `pip install -r requirements.txt`; Start Command `streamlit run app.py --server.port $PORT --server.address 0.0.0.0`; Instance Type Free.

# 2 Part 2: Relation-based argument mining

In this part, we build a complete pipeline for Argument Mining using the Kialo platform. The work is divided into two main steps: scraping and exploring the raw debate data (Question 1), and training a Transformer model to automatically discover and classify relations between these arguments (Question 2).

## 2.1 Question 1: Data Scraping and Exploration

Kialo is a highly structured online debate platform where users construct logical argument trees (Pro/Con) around specific claims. To build our dataset, we inspired our approach from the provided notebook.

**Data Scraping.** Using the public Kialo API, we fetched the metadata and exported the textual structures of the top 250 most active and highly ranked discussions. Because Kialo exports require session authentication, we implemented a custom HTTP session handler injecting browser cookies to successfully download the `.txt` tree structure of each debate.

**Data Exploration and Results.** Once downloaded, we conducted a quick exploratory data analysis (EDA) on the 250 debates to understand the nature of the dataset before feeding it to a model.

- **Debate length and depth:** The debates vary greatly in size, with some containing only a few dozen arguments, while the most active ones contain thousands of deeply nested claims.
- **Thematic Distribution:** We extracted the tags associated with each debate to understand the topics discussed. As shown in Figure 8, the most frequent topics revolve around *Politics*, *Society*, *Religion*, and *Ethics*. This diversity is excellent for training an Argument Mining model, as it ensures the model learns logical structures rather than overfitting to the vocabulary of a single topic.

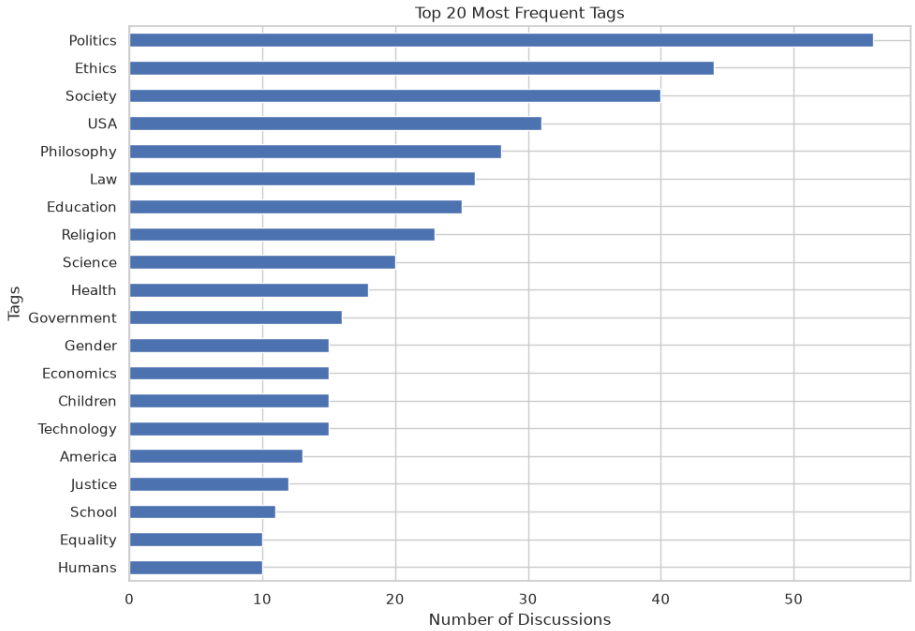



Figure 8: Distribution of the top tags across the 250 scraped Kialo debates.

## 2.2 Question 2: Relation Discovery and Classification

Moving to the predictive task, we aim to automatically identify if two textual arguments are related (*Relation Discovery*) and, if so, determine the nature of this relation (*Support* vs *Attack*). We implemented an end-to-end approach based on a Transformer architecture, drawing inspiration from recent literature, particularly Ruiz-Dolz et al. (2021) [3].

**Dataset Construction.** We parsed the raw `.txt` files to extract parent-child argument pairs.

- **Positive Pairs:** A child node tagged `Pro` to its parent yields a `Support` label. A `Con` tag yields an `Attack` label.
- **Negative Pairs:** To allow the model to discover relations, we generated artificial `No-Relation` pairs by randomly matching arguments from the same debate that do not share a direct link in the Kialo tree.

This yielded a dataset of over 125,000 valid text pairs, naturally balanced across the three classes ( 50% `No-Relation`, 25% `Support`, 25% `Attack`), eliminating the need for artificial resampling.

**Cross-Topic Split and Model Architecture.** To ensure the model learns the logical structure of arguments rather than topic-specific vocabulary, we performed a *Cross-Topic* split (Leave-Debate-Out). We separated the unique debate IDs into 70% Train, 15% Validation, and 15% Test. All pairs belonging to a specific debate were kept strictly within the same set.

Due to computational and hardware constraints on the Google Colab environment, the final RoBERTa model was fine-tuned on a representative, shuffled subset of 10,000 training pairs and 2,000 validation pairs. This proved sufficient to demonstrate the validity of our pipeline and achieve competitive metrics.

The model is RoBERTa-base with a classification head on three classes (`No-Relation`, `Attack`, `Support`). The two arguments of a pair are given together to the tokenizer (`<s> arg1 </s></s> arg2 </s>`), so the model reads both texts at the same time. Table 3 gives the training settings, and Table 4 the scores after each epoch.

| | |
|---|---|
| Model | `roberta-base` |
| Classes | 3 |
| Max length | 512 tokens (truncation) |
| Learning rate | $2 \times 10^{-5}$ |
| Batch size | 16 |
| Epochs | 3 |
| Weight decay | 0.01 |
| Metric | macro-F1 |
| Seed | 42 |
| GPU, time | Colab T4, 56 min |

Table 3: Training settings.

| Epoch | Train loss | Val. loss | Macro-F1 |
|---|---|---|---|
| 1 | 0.840 | 0.695 | 0.459 |
| 2 | 0.608 | 0.603 | 0.719 |
| 3 | 0.481 | 0.606 | 0.742 |

Table 4: Scores after each epoch.

**Results and Comparison with the Literature.** After the training process, our model was evaluated on the unseen Kialo test set. Table 5 presents our results alongside standard baselines reported in recent Argument Mining literature for the task of Argumentative Relation Classification (Support vs. Attack).

| Approach / Authors | Model | Dataset | Macro F1-Score |
|---|---|---|---|
| Stab & Gurevych (2017) [1] | SVM + Features | Persuasive Essays | 0.68 |
| Kobbe et al. (2020) [2] | BERT | Debate portals (Microtexts) | $\sim 0.70$ |
| Ruiz-Dolz et al. (2021) [3] | RoBERTa-base | US2016 Presidential Debates | $\sim 0.70$ |
| Ruiz-Dolz et al. (2021) [3] | RoBERTa-base | Moral Maze (Cross-domain) | 0.61 |
| Jo et al. (2021) [4] | BERT + Soft Logic | Kialo & Debatepedia | $0.68 - 0.74$ |
| **Our Approach** | **RoBERTa-base** | **Kialo (Cross-Topic)** | **0.74** |

Table 5: Comparison of Relation Classification performance with existing literature.

**Analysis.** As shown in Table 5, our performance (**0.74**) aligns with, and potentially exceeds, the standard baselines observed across various argumentative corpora. While traditional models (like SVMs) or Transformers evaluated on noisy cross-domain data (e.g., Moral Maze) struggle to maintain high accuracy, our cross-topic evaluation demonstrates robust generalization. We attribute this success to Kialo's data quality. Unlike transcripts of spoken debates (like US2016) or student essays which often contain rhetorical noise, Kialo provides heavily edited and explicit argumentative language.

**Computational Constraints and Sampling.** Training a Transformer model on the full dataset (over 125,000 pairs) proved highly computationally expensive. Initial attempts using a Google Colab T4 GPU indicated that full training would require nearly 10 hours, and memory constraints (RAM saturation) occasionally disrupted the environment. To overcome these limitations while maintaining the benefits of GPU acceleration, we downsampled the dataset and strictly maintained the cross-topic methodology. We randomly selected a representative subset of 10,000 training pairs, 2,000 validation pairs, and 2,000 test pairs. This pragmatic approach allowed us to successfully leverage the T4 GPU to fine-tune the model for 3 epochs in under an hour, fully validating our Argument Mining pipeline while achieving highly robust metrics.

# 3 Part 3: Visualising acceptability degree spaces for gradual semantics

In the first parts of the course, an argument is either accepted or rejected. A *gradual semantics* is more nuanced: it gives each argument a degree between 0 and 1, which says how strong the argument is once its attackers are taken into account. In this part, we implement the weighted h-categorizer and use it to look at the *acceptability degree space* of a graph: if the weights can be anything between 0 and 1, which degrees can the arguments get? We want to understand how the structure of the graph limits these degrees, and what role the weights play.

## 3.1 Organisation of the code

**Files.** The code of Part 3 is in the folder `part3/`:

- `q1_tools.py`: the weighted h-categorizer (`attack_matrix`, `weighted_h_categorizer`);
- `q2_tools.py`: the functions of Question 2 (sampling, convex hull, heat-maps, comparison of heat-maps, and our two proposals for many arguments);
- `graphs.py`: all the argumentation frameworks we test.
- `q1_hcategorizer.ipynb` and `q2_degree_space.ipynb`: the notebooks. They follow the questions step by step, and each result is followed by a short remark. All the outputs are kept.

**Reproducibility.** The notebooks must be run from top to bottom from inside `part3/`, with the libraries of `requirements.txt` (NumPy, SciPy, Matplotlib, pandas). All the random draws use a fixed seed, so every run gives the same numbers. The full run takes a few minutes, mostly because of one cell that builds a convex hull in 7 dimensions.

**External code.** We did not copy open-source code, but we rely on libraries. The convex hull is computed with `scipy.spatial.ConvexHull`, which uses the Qhull library (`http://www.qhull.org/`), with its default options. To draw the hull in 2D, we followed the example of the SciPy documentation, which joins the points `hull.vertices` in order; we only add the first vertex at the end to close the shape.

### 3.2 Question 1: Weighted h-categorizer

We first implemented the weighted h-categorizer to compute the acceptability degree of each argument.

**Representing the attacks.** We represent the argumentation framework using an attack matrix $M$, where $M[i, j] = 1$ if argument $i$ attacks argument $j$, and 0 otherwise (function `attack_matrix`). This allows us to use matrix multiplication to compute the sum of the attackers' degrees for every argument.

**Computing the degrees.** We implemented the function `weighted_h_categorizer`, which iteratively applies the formula

$$HC_0(a) = w(a), \qquad HC_{k+1}(a) = \frac{w(a)}{1 + \sum_{(b,a)\in R} HC_k(b)}, \qquad \text{stop when } \max_a |HC_{k+1}(a) - HC_k(a)| < \varepsilon.$$

We initialise the degrees with the argument weights and repeat the computation until the maximum change between consecutive iterations is smaller than $\varepsilon = 10^{-6}$, with a limit of 10,000 iterations. The main loop is:

```
HC = W.copy() # start from the weights
for k in range(1, max_iter + 1):
    attack_sum = HC @ M # sum of the attackers' degrees
    HC_next = W / (1 + attack_sum) # h-categorizer formula
    if np.max(np.abs(HC_next - HC)) < epsilon: # convergence check
        return HC_next, k
    HC = HC_next
```

We also included basic input checks to ensure that the matrix and weight vectors have compatible dimensions, that the weights belong to $[0, 1]$, and that $\varepsilon$ is positive.

**Checking the implementation.** We tested the function on eight graphs, including examples from TD3 with and without weights, as well as special cases such as a graph without attacks, an attacker with weight zero, and an argument attacking itself. All computed degrees matched the expected results. We also verified that processing 100,000 weight vectors at once gives the same degrees as processing them individually, confirming that the implementation is suitable for the experiments in Question 2.

### 3.3 Question 2: acceptability degree spaces

The statement asks for a program that takes a framework $(A, R)$, samples 100,000 weight vectors, computes the corresponding degree vectors, and shows them with a convex hull and a heat-map. Figure 9 shows the functions we wrote for this and how the data goes from one to the other. Each subsection below follows one step of the question: what we implemented, what we obtained, and what it tells us about the acceptability degree space.

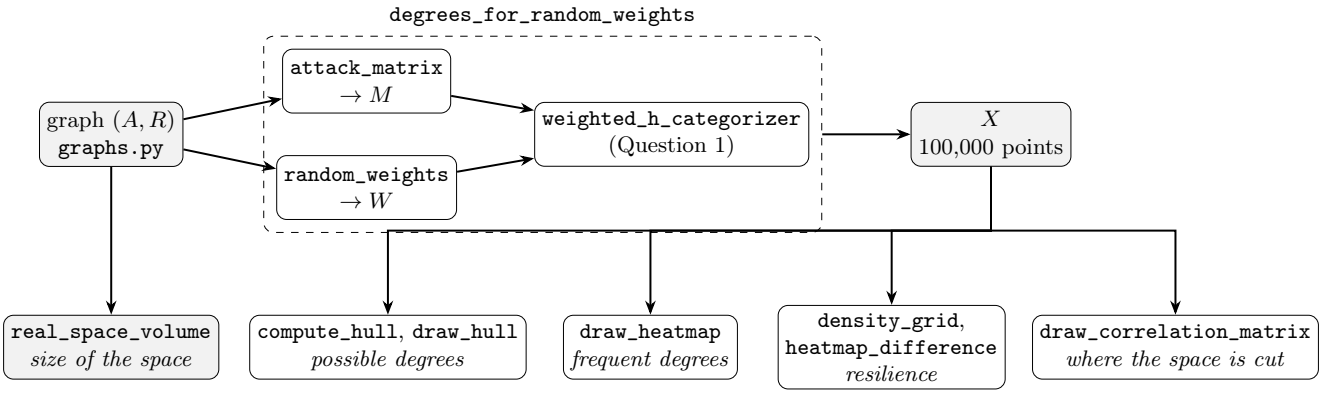



Figure 9: Functions of Question 2 and what each one produces.

#### 3.3.1 Sampling 100,000 weight vectors and computing the degrees

The function `degrees_for_random_weights` takes a framework $(A, R)$, draws 100,000 weight vectors uniformly in $[0, 1]$ (fixed seed), and runs our weighted h-categorizer on all of them at once.

```
def degrees_for_random_weights(arguments, attacks, n_samples=100_000, seed=0):
    M = attack_matrix(arguments, attacks) # the graph (A, R)
    W = random_weights(len(arguments), n_samples, seed) # 100,000 weight vectors
    X, _ = weighted_h_categorizer(M, W) # degrees of every weighted graph
    return W, X
```

The function returns two tables with one row per sample: `W` holds the weights $w_i$ and `X` the degrees $x_i$ (one column per argument). Table 6 shows three rows for $b$ attacks $a$.

| sample | $w(a)$ | $w(b)$ | $Deg(a)$ | $Deg(b)$ |
|---|---|---|---|---|
| 1 | 0.637 | 0.270 | 0.502 | 0.270 |
| 2 | 0.813 | 0.913 | 0.425 | 0.913 |
| 3 | 0.816 | 0.003 | 0.814 | 0.003 |

Table 6: Three of the 100,000 rows for $b$ attacks $a$.

**What this gives us:** each weight vector becomes one point $(Deg(a), Deg(b))$ of the space. The 100,000 points together are our picture of the acceptability degree space.

### 3.3.2 Constructing and displaying the convex hull

**Why a convex hull?** The 100,000 points are only samples: the space itself is the whole region where these points can fall. The convex hull is the smallest convex shape that contains all the points, like an elastic band stretched around the cloud. It turns our points into one region, so it gives an approximation of the acceptability degree space, and we can measure its size.

**How we built it.**

- `compute_hull` calls `scipy.spatial.ConvexHull` on the points. It works in any dimension and gives the corners of the hull, its faces and its size (`hull.volume`: the area in 2D, the volume in 3D).
- `draw_hull_2d` and `draw_hull_3d` draw it: in 2D by joining its corners in order, in 3D as a surface made of its triangular faces.

**Our example: $b$ attacks $a$.** Figure 10 (left) shows the 100,000 points and their hull. The top-right corner is empty: $a$ and $b$ are never strong at the same time, and the hull covers about three quarters of the square (area 0.748). We also notice a thin white strip between the hull and the points: the border of the cloud is curved, while the hull is made of straight lines. So the hull is an estimation of the space, slightly bigger than the real one.

**What this tells us about the space:** the attacks, not the weights, decide which degrees are possible. Each attack puts a ceiling on the attacked argument: even with the highest weight, $a$ cannot go above $1/(1 + Deg(b))$, so when $b$ is strong, $a$ stays low. This ceiling cuts away the region where the arguments are strong together.

### 3.3.3 Applying a heat-map on the convex hull

**Why a heat-map?** The hull gives the shape of the space, but not how the points are spread inside it. A heat-map colours each area by its number of points, so it shows which degrees are frequent.

**How we built it.** `draw_heatmap_2d` and `draw_heatmap_3d` cut the space into small cells (hexagons in 2D, small cubes in 3D), count the points in each cell and colour them: dark = many points, empty cells stay white. We draw the hull on top, so both are on the same figure.

**Our example: $b$ attacks $a$.** Figure 10 (right) shows the heat-map. The weights are drawn uniformly, yet the points are not spread evenly: they pile up at the top left, where $b$ is strong and $a$ is low.

**What this tells us about the space:** the hull shows which degrees are *possible*, the heat-map which ones are *likely*. Because of the ceiling above, when $b$ is strong all the possible weights of $a$ are squeezed into a small range of degrees, so the points pile up there. So $a$ can be strong, but it rarely is.

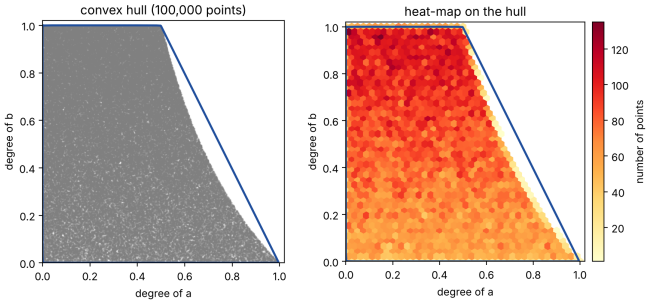



Figure 10: $b$ attacks $a$: convex hull of the 100,000 points (left) and heat-map on the hull (right).

### 3.3.4 Is the heat-map resilient to re-sampling?

The heat-map comes from random weights. If we draw new ones, do we get the same picture, or are we looking at noise? To answer, we kept the same graph and changed only the seed.

**How we visualised it.** We first drew the heat-map for 6 seeds (Figure 11). They look the same: same cut corner, same dense area at the top left. But we wanted to measure it.

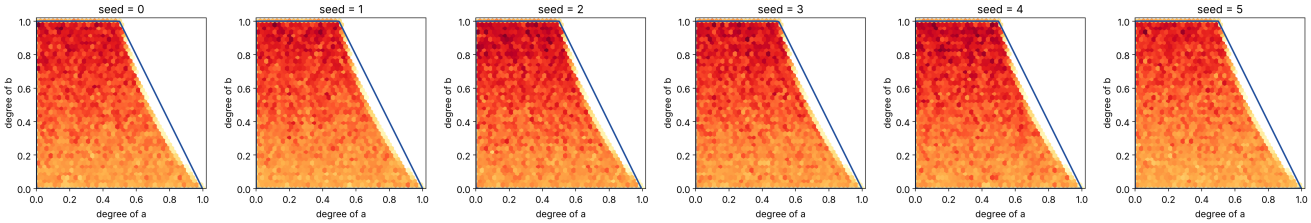



Figure 11: $b$ attacks $a$: heat-maps for 6 different seeds.

**How we measured it.** We wrote one function that turns a heat-map into numbers and one that compares two of them.

- `density_grid` cuts the space of degrees into small cells ($40 \times 40$ in 2D) and gives the share of points in each cell: the concentration we see on the heat-map becomes a value.
- `heatmap_difference` compares the grids $P$ and $Q$ of two seeds: $\frac{1}{2}\sum_{\text{cells}} |P - Q|$ is the share of points that are not in the same cell (0 = identical heat-maps, 1 = completely different).

We ran the test on every graph we used afterwards, with 2, 3 and 4 arguments, comparing all 15 pairs of seeds (Table 7). With more arguments we used $12^3$ and $6^4$ cells instead of $40^2$, so that each cell gets about the same number of points and the results stay comparable.

| Arguments | Graph | mean difference between two seeds |
|---|---|---|
| 2 | no attack / $b \to a$ / $a \leftrightarrow b$ / $a \to a$ | 7.1% / 6.0% / 5.5% / 5.7% |
| 3 | chain / two attackers / 3-cycle | 5.5% / 5.5% / 5.0% |
| 4 | $d \to b \to a$, $c \to a$ | 4.4% |

Table 7: Difference between the heat-maps of two random draws (100,000 samples, mean over 15 pairs of seeds).

**Results.** With 100,000 samples, the heat-map is resilient to re-sampling.

- **Same graph, different seeds:** for every graph, only a small share of the points changes cell from one draw to another (Table 7).
- **Does the number of samples matter?** We also ran the test on $b$ attacks $a$ with 1,000, 10,000, 100,000 and 1,000,000 samples. The difference goes from 53% to 19%, 6% and 2%: the more samples we draw, the more stable the heat-map is. With 100,000 samples, it is stable enough.

**What this tells us about the space:** the dense and empty areas do not depend on the draw, they come from the graph itself. So we can trust what the heat-map shows and interpret it in the next sections.

### 3.3.5 Varying the argumentation frameworks

To see how the attacks change the space, we kept 2 arguments and took four graphs, each with a different kind of attack: no attack (our reference), $b$ attacks $a$, $a$ and $b$ attack each other, and $a$ attacks itself. We ran the same steps on each one (Figure 12) and, to compare them beyond the eye, computed three metrics (Table 8): the hull area (how much of the square is cut), the mean degree of each argument (where it usually ends up) and the share of samples where all degrees are above 0.5 (how often the arguments are strong together).

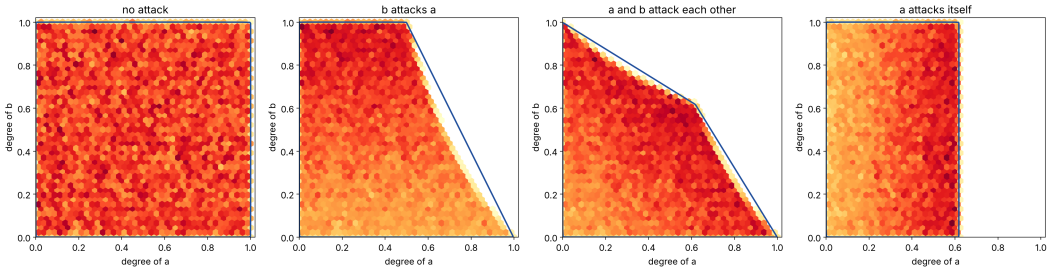



Figure 12: Heat-map and convex hull for four graphs with 2 arguments.

| Graph | hull area | mean $Deg(a)$ | mean $Deg(b)$ | all $> 0.5$ |
|---|---|---|---|---|
| no attack | 1.000 | 0.499 | 0.500 | 24.9% |
| $b \to a$ | 0.748 | 0.346 | 0.500 | 6.2% |
| $a \leftrightarrow b$ | 0.616 | 0.380 | 0.381 | 4.1% |
| $a \to a$ | 0.618 | 0.348 | 0.500 | 12.4% |

Table 8: Metrics of the space for the four graphs with 2 arguments.

**What this tells us about the space:**

1. **The graph decides the shape of the space.** Without attack, the whole square is reachable and filled evenly; every attack cuts away a part of it and makes it rarer for the arguments to be strong together. The weights only choose a point inside this shape.
2. **Each kind of attack cuts the space in its own way.** An attack from another argument cuts the corner where both are strong, a mutual attack cuts it from both sides, and a self-attack puts a fixed ceiling on the argument (never above 0.618, even with weight 1), which gives a rectangle.
3. **An unattacked argument keeps its weight** and covers its whole axis, while an attacked one is pushed down. These are the *maximality* and *weakening* principles of the lecture.

### 3.3.6 Varying the number of arguments

We now add arguments to check two things: do the rules above still hold, and does our method still work?

**3 arguments.** The space is now a volume. We applied the same steps and metrics as with 2 arguments, on three graphs that each add something new (Figure 13, Table 9).

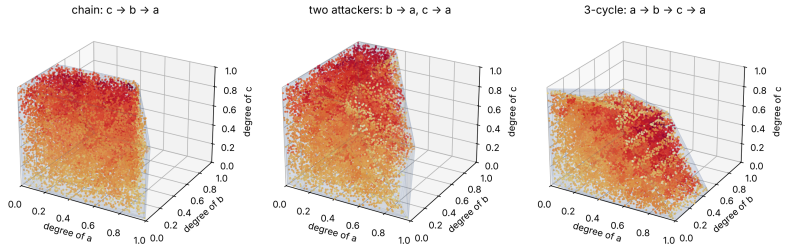



Figure 13: 3D heat-maps (20,000 points shown) and convex hulls for three graphs with 3 arguments.

| Graph | hull volume | mean $Deg(a)$ | mean $Deg(b)$ | mean $Deg(c)$ | all $> 0.5$ |
|---|---|---|---|---|---|
| chain $c \to b \to a$ | 0.589 | 0.382 | 0.345 | 0.500 | 1.4% |
| two attackers $b, c \to a$ | 0.600 | 0.262 | 0.498 | 0.500 | 0.0% |
| 3-cycle | 0.472 | 0.373 | 0.371 | 0.373 | 1.0% |

Table 9: Metrics of the space for the three graphs with 3 arguments (12.5% of samples have all degrees above 0.5 without attack).

**What this tells us about the space:** a third argument shows how the arguments act on each other. In the chain, $c$ weakens the attacker of $a$, so $a$ gets back some room compared with a single attacker: an argument can be *defended* by the attacker of its attacker. With two attackers, $a$ is pushed down further: attacks on the same argument *add up*, which is the *counting* principle of the lecture. The method still works, but a 3D plot is harder to read, so the metrics become more useful.

**4 arguments.** With $d \to b \to a$ (defence) and $c \to a$ (second attacker), the space has 4 dimensions and cannot be drawn. We tried two ideas.

- **3D views** (Figure 14): we look at the space 3 arguments at a time (abc, abd, acd, bcd). Each view looks like a 3-argument graph we already know, so the 4D space is made of the same cuts. But each view forgets one argument: when $a$ is low, we cannot always tell which attacker is responsible.

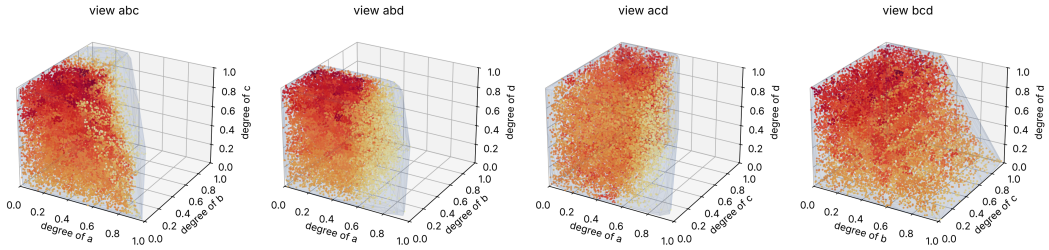



Figure 14: 4 arguments ($d \to b \to a$, $c \to a$): the four 3D views, each one forgets one argument.

- **One more argument as colour** (Figure 15): to fix this, we keep $a$ and $b$ on the axes and use the colour for a third argument, $c$ or $d$. Coloured by $c$, $a$ is only high when $c$ is weak: $c$ limits $a$ directly. Coloured by $d$, the colour follows $b$, not $a$: $d$ limits $b$, and only helps $a$ through $b$.

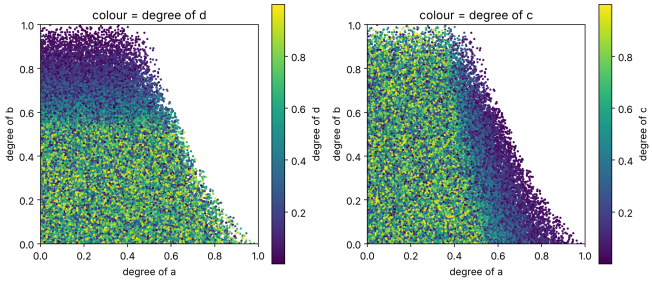



Figure 15: 4 arguments: degree of $a$ against degree of $b$, coloured by the degree of $d$ and $c$.

**What this tells us about the space:** the space keeps shrinking (hull volume 0.46), and $a$ ends up between the case of one attacker and the case of two, because $d$ defends it. Nothing new happens to the space itself; what changes is that we can only see it piece by piece.

**More arguments.** If we keep the method of the statement, we reach a limit (Table 10): the number of 3D views we would need explodes, and the hull becomes too costly to compute, until it runs out of memory. Since we could no longer draw the space, we proposed two simple ways to still answer the same questions about it: *how big is it* and *where is it cut*. We did not use a PCA projection, because its axes mix the arguments and we could no longer read the degree of a given argument.

- **How big is the space?** We turned the formula of Question 1 around: from a degree vector, $w(a) = Deg(a)\,(1 + \sum_{(b,a)\in R} Deg(b))$ gives the only weights that could produce it. We draw random degree vectors and keep those whose weights are all in $[0, 1]$: the share we keep is the volume of the real space (`real_space_volume`). On $b$ attacks $a$ it gives 0.693 against 0.748 for the hull, and it takes less than a second for any number of arguments.
- **Where is the space cut?** We compute the correlation between the degrees of every pair of arguments and colour the matrix (`draw_correlation_matrix`, Figure 16): red when one degree goes down as the other goes up (an attack), blue when they go up together (a defence), white when they are not linked. It only looks at pairs, but it shows which argument cuts the space of which.

| $n$ | 3D views | hull faces | hull time | real volume |
|---|---|---|---|---|
| 2 | 0 | 24 | 0.02 s | 0.693 |
| 4 | 4 | 2,957 | 0.07 s | 0.389 |
| 6 | 20 | 303,275 | 8.9 s | 0.216 |
| 7 | 35 | 3,241,852 | 251 s | – |
| 8 | 56 | out of memory | | 0.120 |
| 15 | 455 | – | | 0.015 |

Table 10: Chains $x_n \to \cdots \to x_1$: hull cost and real volume.

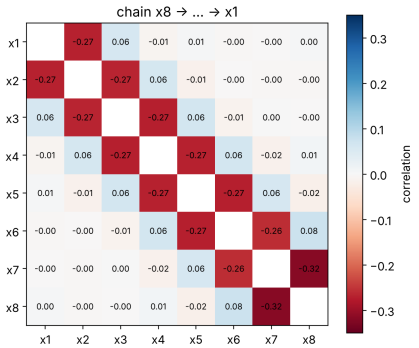



Figure 16: Correlations, chain of 8 arguments.

**What this tells us about the space:** the rules hold for any number of arguments. Each new attack makes the space smaller, and on the chain of 8 the matrix shows that an argument mostly acts on its neighbour (red), helps the argument two steps away (light blue), and has almost no influence beyond.

### 3.3.7 Our analysis

**What visualising the space taught us.** The degree of an argument is not only decided by its weight: the graph sets limits that no weight can cross, and the weights only choose a point inside them. The principles of the lecture become visible in the shape of the space:

- **Maximality:** an unattacked argument keeps its weight and covers its whole axis.
- **Weakening:** each attack cuts away the corner where the attacker and its target are strong together.
- **Counting:** two attackers push the target lower than one.
- **Defence:** the attacker of an attacker gives back some room, but this fades after two steps.

The shape also says more than a number: $a \leftrightarrow b$ and $a \rightarrow a$ have almost the same hull area, but in the first each argument limits the other, while in the second only $a$ is capped.

**Possible and frequent degrees.** The hull shows which degrees are possible, and depends only on the graph. The heat-map shows which ones are frequent, and also depends on how we draw the weights: another distribution than the uniform one would move the dense areas.

**How far we can trust the method.** With 100,000 samples the heat-map no longer depends on the draw, but stable does not mean exact: the convex hull stays too big when the border of the space is curved, which is why we measured the real size with `real_space_volume`. The pictures work up to 3 arguments and piece by piece with 4; beyond that, only numbers remain (volume, correlations), which tell us how big the space is and where it is cut, but not its shape.

## Conclusion

In this report, we worked on three complementary sides of argumentation.

In Part 1, our ABA generator builds the arguments and attacks of any framework, converts it into a non-circular and atomic one, and handles preferences with normal and reverse attacks. Its results are identical to the correction of TD4, and the online application makes it usable without installing anything. The main limit is the size of the conversion: the number of assumptions and arguments grows quickly with the number of non-assumptions. The generator also stops at the attacks; computing the extensions (admissible, preferred, ...) would be the natural next step.

In Part 2, we built a full argument mining pipeline, from the scraping of Kialo debates to the classification of relations. With a cross-topic split, our RoBERTa model reaches a macro F1-score of 0.74 on unseen debates, which is in the range of the results reported in the literature. This comparison should however be read with care: the other works use different datasets and tasks, and our model was trained on a subset of 10,000 pairs because of hardware limits. Training on the full dataset of more than 125,000 pairs would give a more reliable measure.

In Part 3, visualising acceptability degree spaces showed that the degree of an argument does not depend only on its weight: the graph sets limits that no weight can cross, and the weights only choose a point inside them. The principles of the lecture (maximality, weakening, counting, defence) become visible in the shape of the space. The method is stable with 100,000 samples, but the convex hull overestimates the space when its border is curved, and pictures stop being readable beyond three or four arguments. For larger graphs, the real volume and the correlation matrix still tell us how big the space is and where it is cut. A natural extension would be to compare other gradual semantics, or other weight distributions, with the same tools.

Overall, these three parts show the same idea from different angles: whether arguments are built from rules, extracted from text, or given a degree, what we can conclude depends above all on the structure of the attacks between them.

## References


[1] C. Stab, I. Gurevych. Parsing argumentation structures in persuasive essays. *Computational Linguistics*, 43(3), 2017.

[2] J. Kobbe, I. Hulpuş, H. Stuckenschmidt. Unsupervised stance detection for arguments from consequences. In *Proceedings of EMNLP*, 2020.

[3] R. Ruiz-Dolz, J. Alemany, S. Heras, A. García-Fornes. Transformer-based models for automatic identification of argument relations: a cross-domain evaluation. *IEEE Intelligent Systems*, 36(6), 2021.

[4] Y. Jo, S. Bang, C. Reed, E. Hovy. Classifying argumentative relations using logical mechanisms and argumentation schemes. *Transactions of the ACL*, 9, 2021.